SOLSTITIUM

Plan de trabajo, reparto de commits y flujo en GitHub

Periodo	5 – 30 de noviembre de 2026 · 4 sprints
Método	Ágil-incremental
Arquitectura	MVC (Spring Boot + aplicación web)
Repositorio	github.com/Mellozky/SOLSTITIUM · rama **main**
Equipo	Daniel Mendoza · Luis Hernández · Carlos Sandoval · Esteban Vidal

1. Método ágil-incremental

El producto se construye por **incrementos**: cada sprint (una semana) termina con algo que funciona, probado y unido a **main**. Cada incremento se suma al anterior. Si falta tiempo, se deja fuera lo menos importante, nunca el simulador.

- **Backlog:** tareas como *issues* en un tablero de GitHub Projects (Por hacer · En curso · Hecho).
- **Planeación** al iniciar el sprint · **daily** de 15 min · **demo y retro** al cerrar.
- **Facilitador rotativo**: arma el tablero, modera la daily y crea la versión del sprint. S0 Daniel · S1 Esteban · S2 Luis · S3 Carlos.
- **Trabajo por funcionalidad completa:** quien toma una funcionalidad hace su back-end, su pantalla y sus pruebas.
- **Hecho** = funciona, tiene pruebas en verde y está en main.

Calendario



Sprint	Fechas	Se entrega	Versión
0 · Base	5 – 8	Repo, CI, base de datos, entidades, front-end vacío	—
1 · Simulación	9 – 15	Formulario + cálculo completo (RF-01 a RF-06)	v0.1.0
2 · Resultados + admin	16 – 22	Resultados, PDF, login, parámetros, paneles	v0.2.0
3 · Entrega	23 – 30	Pantallas admin, despliegue, pruebas finales, manuales	v1.0.0

Desde el **viernes 27** solo entran correcciones, pruebas y documentación.

2. Patrón MVC

Capa	Dónde	Qué hace
Modelo	model/, service/, repository/	Datos, reglas de negocio y cálculos. Acceso a PostgreSQL.
Vista	frontend/	Pantallas. Solo muestra y envía datos a la API (JSON).
Controlador	controller/	Recibe la petición, valida y llama a un servicio.

```
src/main/java/com/solstitium/solstitium/
├── controller/      endpoints REST
├── service/         lógica de negocio
│   └── calculo/     calculadoras de sombra, técnica, financiera y validador
├── repository/      acceso a datos
├── model/entity/    una clase por tabla
├── model/dto/       datos de entrada y salida de la API
├── config/          seguridad y CORS
└── exception/       errores con mensaje claro
src/main/resources/db/migration/   scripts de Flyway (V1, V2, ...)
frontend/                          aplicación web
```

Reglas

- Flujo: controlador → servicio → repositorio. Nunca al revés ni saltando capas.
- Sin cálculos en controladores ni en el front-end.
- Ningún valor de negocio fijo en el código: todo sale de la base de datos.
- Dinero y factores con **BigDecimal**.
- La API devuelve DTO, no entidades.

3. Trabajo en GitHub

Rama main

main es la rama principal y la única permanente. Es la versión oficial: siempre compila, pasa las pruebas y es la que se despliega. No usamos Pull Requests: cada uno trabaja en su rama y la une a main cuando su tarea funciona.

- Antes de subir a main: traer los cambios de los demás y correr las pruebas.
- Avisar en el grupo al subir algo a main.
- Prohibido forzar (force push) o borrar main.

Ramas y commits

Rama	Uso	Ejemplo
feature/	funcionalidad	feature/sombra
fix/	corrección	fix/redondeo-paneles
test/	pruebas	test/no-funcionales
docs/	documentación	docs/manuales
chore/ · ci/	configuración y despliegue	chore/deploy-render

Mensaje: `tipo(ámbito): qué hace` → `feat(calculo): factor de sombra (RF-03)`. Tipos: feat, fix, test, docs, style, refactor, chore, ci. Commits pequeños.

Ciclo de una tarea

Paso	GitHub Desktop	Terminal
1. Actualizar main	Current branch: main → Fetch / Pull origin	`git switch main && git pull`
2. Crear rama	Current branch → New branch (basada en main)	`git switch -c feature/sombra`
3. Commits	Marcar archivos, escribir mensaje → Commit	`git commit -m "feat(...): ..."`
4. Ir a main y actualizar	Current branch: main → Pull origin	`git switch main && git pull`
5. Unir la rama	Branch → Merge into current branch → elegir la rama	`git merge feature/sombra`
6. Probar	—	`./mvnw verify`
7. Subir y avisar	Push origin · avisar en el grupo	`git push`
8. Borrar la rama	Branch → Delete	`git branch -d feature/sombra`

- Si al unir hay conflictos, resolverlos con quien hizo el otro cambio.
- Ramas cortas: 1 a 3 días. Mientras más tiempo vive una rama, más conflictos.
- Si main se rompe, arreglarlo es lo primero.

4. Reparto de commits

Cada fila es un commit; las filas con la misma rama se unen a main juntas. Cada integrante hace una funcionalidad completa (back-end, pantalla y pruebas).

Commit inicial (Daniel): `chore: proyecto base Spring Boot, README y plan de trabajo`

Sprint 0 · Base (5 – 8 nov)

Todos pueden clonar, compilar y probar.

#	Integrante	Rama	Commit
1	**Daniel Mendoza**	`ci/verificacion`	`ci: GitHub Actions con ./mvnw verify y PostgreSQL en cada push`
2	**Daniel Mendoza**	`ci/verificacion`	`ci: build del front-end en cada push`
3	**Daniel Mendoza**	`ci/verificacion`	`feat(bd): migración V2 con datos de prueba del ERS`
4	**Luis Hernández**	`feature/esquema-bd`	`feat(bd): migración V1 con las 11 tablas`
5	**Luis Hernández**	`feature/esquema-bd`	`feat(bd): restricciones e índices únicos`
6	**Luis Hernández**	`feature/esquema-bd`	`feat(repositorio): cargar todos los parámetros (ConjuntoParametros)`
7	**Carlos Sandoval**	`feature/frontend-base`	`chore(front): proyecto React + Vite en frontend/`
8	**Carlos Sandoval**	`feature/frontend-base`	`feat(front): layout, rutas y estilos del prototipo Figma`
9	**Carlos Sandoval**	`feature/frontend-base`	`feat(front): cliente HTTP para /api`
10	**Esteban Vidal**	`feature/entidades`	`feat(modelo): entidades JPA alineadas a la BD`
11	**Esteban Vidal**	`feature/entidades`	`feat(modelo): mapeo del enum tipo_ajuste_estrato`
12	**Esteban Vidal**	`feature/entidades`	`feat(repositorio): repositorios Spring Data`

Sprint 1 · Simulación (9 – 15 nov) → v0.1.0

El usuario llena el formulario y recibe el resultado.

#	Integrante	Rama	Commit
1	**Esteban Vidal**	`feature/consumo-tarifa`	`feat(dto): solicitud de simulación con validaciones (RF-01, RF-02)`
2	**Esteban Vidal**	`feature/consumo-tarifa`	`feat(calculo): tarifa por estrato con subsistencia`
3	**Esteban Vidal**	`feature/consumo-tarifa`	`feat(front): paso 1, consumos y factura`
4	**Esteban Vidal**	`feature/consumo-tarifa`	`feat(front): paso 2, estrato y área`
5	**Esteban Vidal**	`feature/consumo-tarifa`	`test: validación de entrada CV-01..CV-14`
6	**Luis Hernández**	`feature/sombra`	`feat(calculo): factor y tramo de sombra (RF-03)`
7	**Luis Hernández**	`feature/sombra`	`feat(front): paso 3, sombra por franja`
8	**Luis Hernández**	`feature/sombra`	`test(calculo): sombra CP-01..CP-08 y CV-15`

#	Integrante	Rama	Commit
9	Carlos Sandoval	feature/dimensionamiento	feat(calculo): paneles, potencia e inversor (RF-04)
10	Carlos Sandoval	feature/dimensionamiento	test(calculo): dimensionamiento CP-01..CP-09 y CV-29
11	Carlos Sandoval	feature/api-simulacion	feat(api): POST /api/simulaciones
12	Carlos Sandoval	feature/api-simulacion	feat(api): errores con mensajes en español
13	Daniel Mendoza	feature/financiero	feat(calculo): generación, ahorro y cobertura (RF-05)
14	Daniel Mendoza	feature/financiero	feat(calculo): payback, ROI y viabilidad (RF-06)
15	Daniel Mendoza	feature/financiero	feat(simulacion): recomendación y escenario sin sombra
16	Daniel Mendoza	feature/financiero	test(calculo): CP-01..CP-11 completos (±1 %)

Sprint 2 · Resultados + admin (16 – 22 nov) → v0.2.0

Resultados y PDF; el administrador entra y cambia datos.

#	Integrante	Rama	Commit
1	Daniel Mendoza	feature/resultados	feat(front): pantalla de resultados y recomendación
2	Daniel Mendoza	feature/resultados	feat(front): escenario sin sombra y advertencias
3	Daniel Mendoza	feature/resultados	feat(front): exportar a PDF (CU-05)
4	Luis Hernández	feature/login	feat(seguridad): sesión de servidor y BCrypt
5	Luis Hernández	feature/login	feat(auth): login y logout de administrador (CU-06)
6	Luis Hernández	feature/login	feat(front): pantalla de login
7	Luis Hernández	feature/login	test: login y acceso sin sesión CV-17..CV-19
8	Esteban Vidal	feature/parametros	feat(parametros): consultar parámetros por sección
9	Esteban Vidal	feature/parametros	feat(calculo): validador de rangos y coherencia
10	Esteban Vidal	feature/parametros	feat(parametros): guardar + bitácora en una transacción (CU-15)
11	Esteban Vidal	feature/parametros	test: CV-20..CV-23, CV-27 y CV-28
12	Carlos Sandoval	feature/paneles	feat(paneles): registrar, editar, activar y desactivar (CU-13)
13	Carlos Sandoval	feature/paneles	feat(paneles): cambiar panel de referencia (CU-14)
14	Carlos Sandoval	feature/paneles	feat(front): catálogo de paneles
15	Carlos Sandoval	feature/paneles	test: reglas de paneles CV-24..CV-26

Sprint 3 · Entrega (23 – 30 nov) → v1.0.0

Desplegado por HTTPS, probado y documentado.

#	Integrante	Rama	Commit
1	**Esteban Vidal**	`feature/pantallas-parametros`	`feat(front): tarifas, estratos y parámetros técnicos (CU-08, CU-09)`
2	**Esteban Vidal**	`feature/pantallas-parametros`	`feat(front): sombreado, costos y rangos (CU-10..CU-12)`
3	**Esteban Vidal**	`feature/pantallas-parametros`	`docs: manual de administrador`
4	**Luis Hernández**	`chore/deploy-render`	`feat(config): perfil de producción con variables de entorno`
5	**Luis Hernández**	`chore/deploy-render`	`chore(deploy): API en Render con PostgreSQL y HTTPS`
6	**Luis Hernández**	`chore/deploy-render`	`fix: correcciones reportadas en pruebas`
7	**Luis Hernández**	`chore/deploy-render`	`docs: notas de versión v1.0.0`
8	**Carlos Sandoval**	`chore/deploy-vercel`	`feat(bitacora): consulta y pantalla de bitácora (CU-07)`
9	**Carlos Sandoval**	`chore/deploy-vercel`	`style(front): diseño adaptable desde 360 px`
10	**Carlos Sandoval**	`chore/deploy-vercel`	`chore(deploy): Vercel con /api redirigido a Render`
11	**Daniel Mendoza**	`test/no-funcionales`	`test(nf): rendimiento y seguridad NF-01, NF-03, NF-04`
12	**Daniel Mendoza**	`test/no-funcionales`	`test(e2e): regresión completa en el despliegue`
13	**Daniel Mendoza**	`test/no-funcionales`	`docs: manual de usuario`
14	**Daniel Mendoza**	`test/no-funcionales`	`docs(qa): informe de pruebas`

Total por integrante

Integrante	Commits	Funcionalidades
Daniel Mendoza	14 + inicial	CI, cálculo financiero, resultados y PDF, pruebas finales
Luis Hernández	14	Base de datos, sombra, login, despliegue de la API
Carlos Sandoval	14	Front-end base, dimensionamiento, API de simulación, paneles, despliegue web
Esteban Vidal	15	Entidades, consumo y tarifa, parámetros y bitácora, pantallas admin

Es una guía: si una tarea pide más commits pequeños, mejor.

5. A tener en cuenta

Antes de empezar

- Aceptar la invitación al repositorio (la envía Daniel desde Settings → Collaborators).
- Usar en Git el **mismo correo de su cuenta de GitHub**; si no, los commits no salen a su nombre.
- Instalar **JDK 21** (la versión del proyecto).
- Tener PostgreSQL con una base `solstitium`. Sin ella, las pruebas fallan.
- Clonar desde GitHub Desktop (File → Clone repository) o con git clone.

Técnico

- Los ids de las entidades deben coincidir con el tipo de la BD (Short/Integer, no Long), o la app no arranca.
- El enum de PostgreSQL necesita un mapeo explícito en JPA.
- Cambiar panel de referencia: quitar la marca al anterior antes de ponerla al nuevo, en la misma transacción.
- Guardar parámetros y bitácora en una sola transacción: si falla una, no se guarda nada.
- Casos borde: sombra total → margen máximo; 0 paneles → "sin espacio"; ahorro 0 → "no recupera".
- Una migración de Flyway que ya está en main no se edita: se crea otra (V3, V4…).
- Front-end y API en dominios distintos: Vercel debe redirigir /api a Render para que funcione la sesión.
- Nada de contraseñas en el código: el repositorio es público. Usar variables de entorno.

Despliegue gratuito

- Render apaga el servicio si no se usa; la primera petición tarda cerca de un minuto. Abrir la app antes de cada demo.
- La base de datos gratuita de Render vence tras un tiempo (hoy ~30 días): crearla en el sprint 3.
- En Vercel, la carpeta raíz es `frontend`; en Render, la raíz del repositorio.

Proceso

- Nunca trabajar en main. Actualizarla antes de crear cada rama.
- No subir `target/`, `node_modules/`, `.idea/` ni `.env`.
- Si las pruebas de main fallan, arreglarlo es lo primero.
- Avisar atrasos en la daily del mismo día.
- Son 4 semanas en vez de las 10 del ERS: si no alcanza, se recortan primero las pantallas de administración.